A Study of Lightweight Post-Deployment Adaptation for Sequential Recommendation

Nayeema Nonta

1. Introduction

Sequential recommendation learns from past interaction sequences to predict a user's next action. In deployment, however, future behaviour may not match the training period. User interests evolve, local transition patterns shift, and the short-range context around the next interaction may change over time. As a result, a model trained on an earlier period may lose ranking accuracy on a later one.

This gap matters in practice because recovering performance through frequent retraining or continual maintenance can require substantial data, compute, and engineering effort. Most prior work on preference drift in sequential recommendation studies a different setting. Recent methods typically address drift within the model or training pipeline through drift-point detection, time-aware modelling, evolving-interest representations, or related drift-aware mechanisms (Sritrakool & Maneeroj, 2021; Maneeroj & Sritrakool, 2022; Li et al., 2020; Chen et al., 2021; Chi et al., 2022; Shao et al., 2023; Lin et al., 2025; Han et al., 2024; Kang et al., 2025; Hao et al., 2026). Continual and incremental recommendation is closer to deployment realism, but it usually assumes repeated updates over a stream of future data rather than a single correction to an already trained backbone (Lee et al., 2025; Song et al., 2026). In contrast, we study a practical setting where a model trained on past data is deployed under temporal shift and adapted with limited future data (Figure 1).

We study this setting in SASRec (Kang & McAuley, 2018), a widely used sequential recommender with a simple and transparent prediction pipeline. Its relative simplicity makes it a useful testbed for studying where lightweight adaptation is most effective under temporal drift.

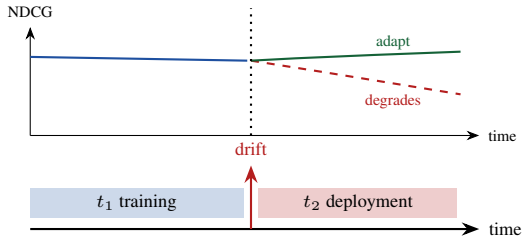


Figure 1. Sequential recommenders trained on historical behaviour (t_1) can degrade after deployment when the future distribution (t_2) shifts. Lightweight post-deployment adaptation aims to recover ranking quality without full retraining.

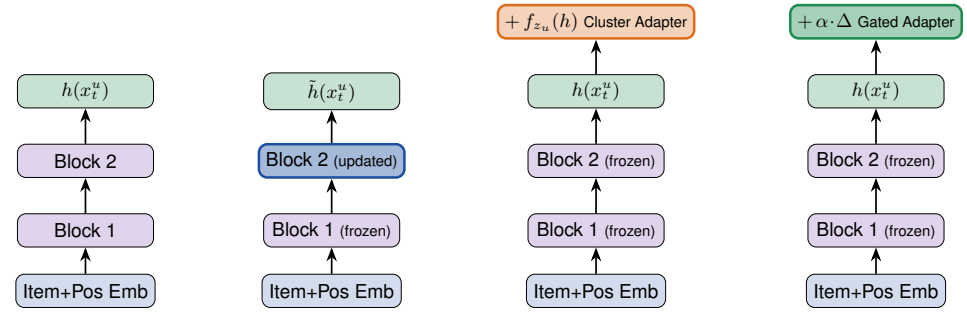
We compare two intervention types. Weight-space adaptation updates the encoder itself, whereas activation-space adaptation keeps the SASRec backbone fixed and modifies only the sequence representation used for scoring. Recent recommender work shows that post-hoc intervention on learned representations can support alignment, bias control, and controllable recommendation (Ahmadov & Mansoury, 2026; Schirmer et al., 2026; Klenitskiy et al., 2025), making activation-space adaptation a plausible option. Related work in language models likewise shows that targeted representation-level interventions can support efficient adaptation (Lai et al., 2025; Wu et al., 2024; Yin et al., 2024). Comparing these intervention types helps distinguish whether post-deployment degradation is better corrected by updating the final sequence representation or by adapting the sequence encoder itself.

To make this comparison interpretable, we evaluate adaptation in a fixed-catalogue overlap regime so that future degradation is not trivially dominated by unseen items. This study provides a controlled comparison of lightweight post-deployment adaptation methods for sequential recommendation. Under the same backbone, loss, and temporal split design, we assess both how much lost next-item ranking performance these methods can recover and what their relative performance suggests about where temporal drift is most effectively corrected in a sequential recommender.

2. Approach

2.1. Problem setup

We study post-deployment adaptation under a two-period protocol. A SASRec backbone is first pretrained on an earlier period t_1, then adapted on a small split from a later period t_2, and finally evaluated on a held-out t_2 test split. We use a fixed-catalogue overlap setting, so future interactions are restricted to items already present in the historical vocabulary. This isolates temporal behavioural shift from pure catalogue expansion.



(a) Shared backbone. (b) Last-block fine-tuning. (c) Prototype steering. (d) Context-gated adapter.

Figure 2. Lightweight post-deployment adaptation methods studied in this work. (a) The shared SASRec backbone. (b) Last-block fine-tuning updates the final encoder block. (c) Prototype steering keeps the backbone frozen and adds a cluster-specific residual correction in activation space. (d) The context-gated adapter keeps the backbone frozen and applies a shared residual correction scaled by an input-dependent gate.

Table 1. Description of Key Symbols.

Symbol	Description
$\mathcal{I}$	item catalog
S^u	interaction sequence of user u
x_t^u	prefix used to predict the next item
i_t^u	observed next item after prefix x_t^u
n	maximum sequence length
d	backbone hidden dimension
r	adapter bottleneck dimension
K	number of prototype clusters
$h(x_t^u) \in \mathbb{R}^d$	backbone hidden state for prefix x_t^u
$\tilde{h}(x_t^u) \in \mathbb{R}^d$	adapted hidden state
$E(j) \in \mathbb{R}^d$	embedding of item j
z_u	cluster assignment for user u
$\Delta(x_t^u) \in \mathbb{R}^d$	residual correction vector
$\alpha(x_t^u) \in (0,1)$	scalar gate for the correction magnitude
$W_1 \in \mathbb{R}^{r\times d}$, $W_2 \in \mathbb{R}^{d\times r}$	bottleneck adapter weights
$\rho(\cdot)$	GELU activation

Let $S^u = (i_1^u, \ldots, i_{T_u}^u)$ denote the interaction sequence of user u. For a prefix $x_t^u = (i_1^u, \ldots, i_{t-1}^u)$, the target is the next item i_t^u. Table 1 summarizes the notation used throughout this section.

2.2. Backbone model

Our backbone is SASRec, a self-attentive sequential recommender for next-item prediction (Kang & McAuley, 2018). Each input prefix is truncated to length n and left-padded when needed. Item embeddings and learned positional embeddings are summed, then passed through a stack of causal self-attention blocks (Figure 2a). Each block contains masked self-attention, a point-wise feed-forward network, residual connections, layer normalization, and dropout. The causal mask prevents access to future items. Given prefix x_t^u, the backbone produces a hidden state $h(x_t^u) \in \mathbb{R}^d$. Candidate item j is scored by

$$s(j \mid x_t^u) = h(x_t^u)^\top E(j).$$

We use a lightweight SASRec configuration with hidden size 64, two self-attention blocks, two attention heads, and maximum sequence length 128 following (Klenitskiy et al., 2024). Additional details in Appendix A.

Backbone training. The backbone is trained on t_1 with full-vocabulary cross-entropy (CE):

$$\mathcal{L}_{\mathrm{CE}}(x_t^u, i_t^u) = -\log \frac{\exp\big(h(x_t^u)^\top E(i_t^u)\big)}{\sum_{j\in\mathcal{I}} \exp(h(x_t^u)^\top E(j))}.$$

We use full-vocabulary CE rather than sampled-negative objectives because it provides stronger gradient signal over the full catalogue.

2.3. Adaptation methods

After training on t_1, we adapt the model on the t_2 adaptation split. All methods use the same scoring rule

$$s(j \mid x_t^u) = \tilde{h}(x_t^u)^\top E(j),$$

and differ only in how the adapted state $\tilde{h}(x_t^u)$ is produced.

Last-block fine-tuning. Last-block fine-tuning is a weight-space method. We freeze all backbone parameters except those in the final SASRec block (Figure 2b). The adapted state $\tilde{h}(x_t^u)$ is the output of this partially updated backbone.

The motivation is that later blocks operate on already contextualized sequence representations, so updating only the final block targets higher-level sequence processing while keeping earlier item and positional encoding fixed.

Prototype steering. Prototype steering is an activation-space method with a frozen backbone (Figure 2c). It applies a cluster-specific residual correction to the SASRec hidden state:

$$\tilde{h}(x_t^u) = h(x_t^u) + f_{z_u}(h(x_t^u)),$$

where $z_u \in \{0, \ldots, K-1\}$ is the cluster assignment of user u, and

$$f_{z_u}(h) = W_{2,z_u}\, \rho(W_{1,z_u} h + b_{1,z_u}) + b_{2,z_u}.$$

The motivation is that temporal drift may differ across users, so a single shared correction may be too coarse. Prototype steering instead learns one small adapter per cluster while keeping the backbone fixed.

User clusters are computed offline with KMeans on drift features derived from the historical-to-future shift: item-profile cosine distance, hidden-state cosine distance, top-k item Jaccard drift, and the historical and future sequence lengths. All features are standardized before clustering. Full details are given in Appendix B.6.

Context-gated adapter. The context-gated adapter is also an activation-space method with a frozen backbone, but it does not use user clusters. Instead, it applies a shared residual correction scaled by an input-dependent gate:

$$\tilde{h}(x_t^u) = h(x_t^u) + \alpha(x_t^u)\, \Delta(x_t^u),$$

where

$$\Delta(x_t^u) = W_2\, \rho(W_1 h(x_t^u) + b_1) + b_2,$$
$$\alpha(x_t^u) = \sigma(w^\top h(x_t^u) + b).$$

Prototype steering provides discrete user-level specialization through cluster-specific adapters. The context-gated adapter instead provides continuous input-dependent modulation through a shared residual and scalar gate.

Adaptation loss. All adaptation methods are trained with the same full-vocabulary cross-entropy objective:

$$\mathcal{L}_{\text{adapt_CE}}(x_t^u, i_t^u) = -\log \frac{\exp\left(\tilde{h}(x_t^u)^\top E(i_t^u)\right)}{\sum_{j \in \mathcal{I}} \exp\left(\tilde{h}(x_t^u)^\top E(j)\right)}.$$

2.4. Evaluation

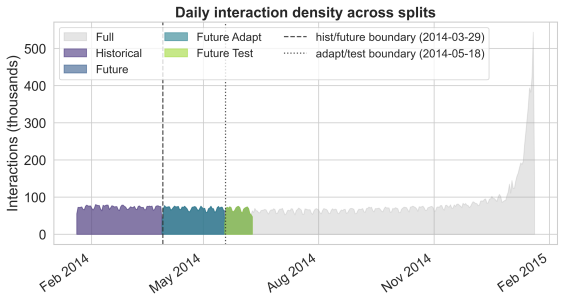
We evaluate all methods under the same post-deployment protocol: pretrain on t_1, adapt on the t_2 adaptation split, and evaluate on a held-out t_2 test split.

At test time, the ground-truth next item is ranked against 100 sampled negatives. We use the same sampled candidate sets for the baseline and adapted models, with sampling fixed by seed 42, so comparisons are paired and directly comparable. We report Hit Rate, NDCG, and MRR based on the target rank within this sampled candidate set.

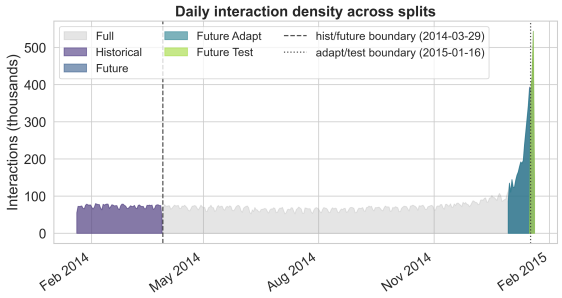
3. Experimental setup

Dataset. We evaluate on 30Music (Turrin et al., 2015), a large-scale music listening dataset with long user histories. It is well suited to our setting because it exhibits strong sequential structure (Klenitskiy et al., 2024).

Temporal regimes. We construct two temporal adaptation regimes from the timestamp-sorted interaction log. In both cases, the earlier period t_1 is used to train the backbone, and the later period t_2 is used for adaptation and evaluation. In the **contiguous** split, t_2 is the block immediately following t_1, yielding a shorter-horizon adaptation setting (Figure 3a). In the **tail** split, t_2 is taken from the chronological tail of the dataset, producing a larger temporal gap and a stronger shift (Figure 3b). Each t_2 block is then divided chronologically into a 70% train split and a 30% test split.



(a) Contiguous split.



(b) Tail split.

Figure 3. Daily interaction density across the two temporal split types. The contiguous split uses a future block immediately following the historical block, while the tail split uses a future block drawn from the chronological end of the dataset.

Overlap-item regime. Many items in t_2 do not appear in the t_1 training catalogue. In preliminary analysis, these out-of-vocabulary items dominated the performance drop. To isolate behavioural drift, we evaluate in an overlap-item regime: after iterative 5-core filtering on t_1, we restrict t_2 to items that remain in the filtered t_1 catalogue.

High-drift user subset. Even within the overlap-item regime, many users remain relatively stable across periods. We therefore rank users by drift between their t_1 and t_2 histories and retain the top 10%. The drift score combines item-profile change, hidden-state change, and top-k item change.

Final evaluation subset. The final evaluation pipeline is: construct temporal splits, apply 5-core filtering to t_1, restrict t_2 to overlap items, score users by temporal drift, retain the top 10% high-drift users, and re-apply 5-core filtering before training and evaluation. Full preprocessing details are given in Appendix B. We also report a sequential-structure sanity check on the filtered subsets in Appendix C, which verifies that overlap filtering preserves substantial order-dependent signal.

4. Results

4.1. Main results

We first evaluate all methods on the high-drift user subset, which is the target setting for post-deployment adaptation. Results are reported in Table 2. Hyperparameter tuning results are reported in Figures 8–11, in Appendix D.

All lightweight adapters improve over the frozen SASRec backbone on both temporal regimes. On the **contiguous** split, Prototype Steering gives the largest gains and clearly outperforms both weight-space baselines and the other adapters. On the **tail** split, where the temporal gap is larger, the gap between methods narrows and all three lightweight methods perform similarly.

A key result is that **Full Fine-Tuning is outperformed by all lightweight adapters**, despite updating the entire model. We attribute this to two effects. First, adaptation is performed on a small and biased subset of high-drift users, so updating 12.4M parameters encourages overfitting to drift-specific sequences rather than learning a correction that generalizes. Second, full fine-tuning can overwrite attention weights and item embeddings that encode broadly useful transition patterns. Even drifted users still depend on these patterns, so changing the full backbone can hurt future ranking performance. In contrast, lightweight adapters preserve the pretrained backbone and apply a constrained correction.

Overall, these results suggest that **parameter efficiency acts as a regularizer, not just a cost saving**. Under limited adaptation data, restricting the update space improves generalization.

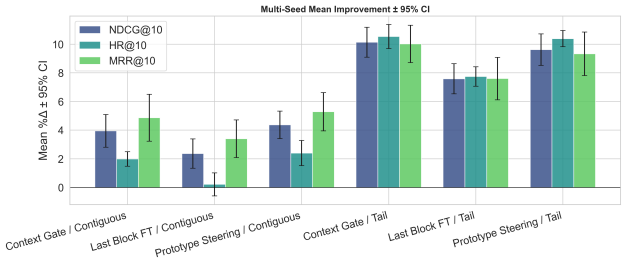


Figure 4. Mean NDCG@10 improvement ± 95% confidence interval across 10 random seeds for each method and split.

4.2. Stability across random seeds

We next test whether these gains are robust across random initialization. Each method is run with 10 random seeds (0–9), and we report mean improvements with 95% confidence intervals in Figure 4. Table 3 shows that the gains are stable across runs and statistically significant for nearly all method–metric pairs. The only exception is HR@10 for Last Block FT on the contiguous split. Overall, these results confirm that the gains from lightweight adaptation are robust and not an artifact of a particular random seed.

Table 3. Multi-seed stability (10 seeds). Mean ± 95% CI of % improvement. $^{***}p < 0.01$, † = adapter only.

Method	NDCG@10	HR@10	MRR@10
Contiguous Split			
Last Block FT	$2.37 \pm 1.02^{**}$	0.22 ± 0.80	$3.40 \pm 1.31^{***}$
Prototype Steering†	$4.37 \pm 0.96^{***}$	$2.40 \pm 0.88^{***}$	$5.29 \pm 1.34^{***}$
Context Gate†	$3.95 \pm 1.14^{***}$	$1.99 \pm 0.51^{***}$	$4.87 \pm 1.64^{***}$
Tail Split			
Last Block FT	$7.59 \pm 1.05^{***}$	$7.74 \pm 0.68^{***}$	$7.60 \pm 1.48^{***}$
Prototype Steering†	$9.62 \pm 1.10^{***}$	$10.39 \pm 0.56^{***}$	$9.33 \pm 1.52^{***}$
Context Gate†	$10.14 \pm 1.04^{***}$	$10.54 \pm 0.84^{***}$	$10.02 \pm 1.30^{***}$

4.3. Effect of cluster size K

The clearest hyperparameter effect appears in the number of clusters K for Prototype Steering. On the **contiguous** split, performance is best at $K = 1$ and declines as K increases. This suggests that, under a shorter temporal gap, the high-drift users form a relatively cohesive group and do not benefit from finer partitioning. On the **tail** split, the pattern reverses: performance peaks at a moderate cluster count and then declines. This suggests that drift becomes more heterogeneous as the temporal gap grows, so cluster-

Table 2. Adaptation results on high-drift users. **Bold** = best adapted value per split per metric. Baseline is the frozen SASRec backbone evaluated on the same candidate sets.

	Contiguous Split									Tail Split								
	NDCG@10			HR@10			MRR@10			NDCG@10			HR@10			MRR@10		
Method	Base	Adpt	%Δ	Base	Adpt	%Δ	Base	Adpt	%Δ	Base	Adpt	%Δ	Base	Adpt	%Δ	Base	Adpt	%Δ
Full FT	0.5874	0.6299	**+7.25%**	0.8419	0.8462	**+0.51%**	0.5072	0.5609	**+10.57%**	0.4676	0.4975	**+6.39%**	0.7060	0.7010	-0.71%	0.3934	0.4333	**+10.12%**
Last Block FT	0.5874	0.6137	**+4.48%**	0.8419	0.8675	**+3.05%**	0.5072	0.5337	**+5.22%**	0.4676	0.5076	**+8.55%**	0.7060	0.7487	**+6.05%**	0.3934	0.4326	**+9.96%**
Context Gate (no gate)	0.5874	0.6142	**+4.56%**	0.8419	0.8675	**+3.05%**	0.5072	0.5338	**+5.24%**	0.4676	0.5139	**+9.90%**	0.7060	**0.7663**	**+8.54%**	0.3934	0.4347	**+10.48%**
Context Gate	0.5874	0.6291	**+7.10%**	0.8419	0.8718	**+3.55%**	0.5072	0.5526	**+8.94%**	0.4676	0.5162	**+10.39%**	0.7060	**0.7663**	**+8.54%**	0.3934	0.4377	**+11.25%**
Prototype Steering	0.5874	**0.7313**	**+24.49%**	0.8419	**0.9017**	**+7.11%**	0.5072	**0.6775**	**+33.56%**	0.4676	**0.5163**	**+10.41%**	0.7060	0.7613	**+7.83%**	0.3934	**0.4399**	**+11.81%**

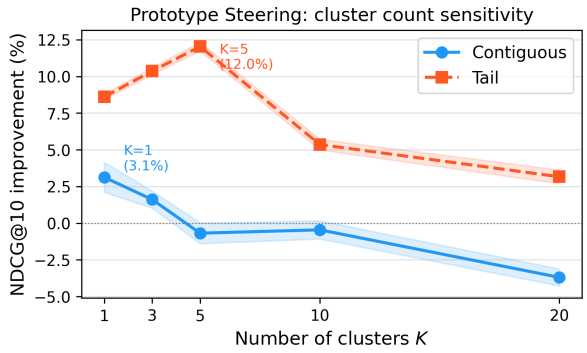


Figure 5. NDCG@10 improvement (%) of Prototype Steering as a function of cluster count K, averaged over all other hyperparameter settings.

specific corrections are more useful, but only up to the point where each cluster still has enough data. Cluster sizes are reported in Appendix B.7, Table 5.

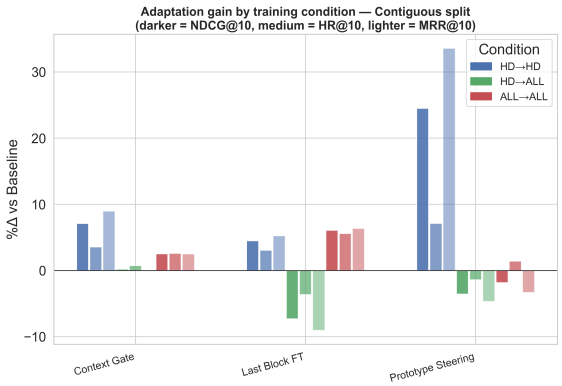
4.4. Generalization beyond high-drift users

The main results target high-drift users by design. To test whether adapters trained on this subset transfer to the broader population, we evaluate three conditions: HD→HD, HD→ALL, and ALL→ALL. Figure 6 summarizes all three conditions for NDCG@10.

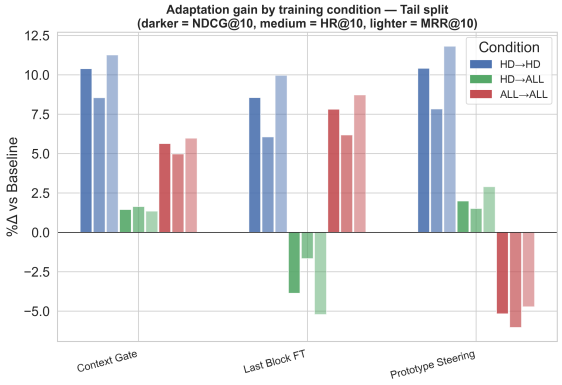
Context Gate shows the strongest robustness. When trained on high-drift users and evaluated on all users, it causes little to no degradation, and when trained on the full population it improves further. This suggests that its gated residual correction can adapt to drifted users without sharply disrupting stable ones.

Last Block FT shows a different pattern. When trained only on high-drift users, it harms performance on the full population. However, when trained on all users, it becomes the strongest method in the general-population setting. This indicates that weight-space adaptation has enough capacity to work well, but only when the training distribution is representative.

Prototype Steering is the most specialized. It performs best in the high-drift setting but does not transfer as well to the full population, especially when trained on all users. We note, however, that due to compute constraints its hyperparameters were not tuned separately for the ALL setting.



(a) Contiguous split.



(b) Tail split.

Figure 6. NDCG@10 improvement under three evaluation conditions: *HD→HD* (targeted adaptation), *HD→ALL* (generalization to full population), and *ALL→ALL* (global adaptation). Context Gate shows negligible collateral impact on stable users (HD→ALL $\approx$ 0), whereas Last Block FT degrades stable-user performance by up to 7.3% on the contiguous split.

4.5. Computational efficiency

We finally compare parameter count, memory use, and training time.

The activation-space methods are dramatically smaller than Full Fine-Tuning and substantially smaller than Last Block FT. Despite this, they match or exceed the heavier baselines in ranking improvement. They also train faster, with Context Gate offering the best overall efficiency.

This makes the main trade-off clear. The strongest adaptation gains do not require updating the full backbone. Instead, small corrections to the learned sequence representation are often sufficient. In this setting, parameter efficiency is not only a computational advantage, but also part of why these methods generalize well.

Context Gate also converges quickly, reaching near-peak performance after only 10 epochs on the contiguous split, whereas Last Block FT benefits from substantially longer training. This makes Context Gate the most attractive method when both robustness and deployment efficiency matter.

Table 4. Computational overhead and performance. Memory/Params from contiguous split. C: Contiguous, T: Tail.

Method	**Params**	**Mem (MB)**	**Time (s)**	**C ($\Delta\%$)**	**T ($\Delta\%$)**
Full FT	12.4M	1381	17.8	+7.25	+6.39
Last Block FT	25.3K	1240	36.7	+4.48	+8.55
Context Gate	1.2K	1239	6.6	+7.10	+10.39
Prototype Steering	2.1K	1240	14.7	+24.49	+10.41

5. Conclusion

We studied lightweight post-deployment adaptation for sequential recommendation under temporal shift. Across both temporal regimes, activation-space methods consistently matched or outperformed heavier weight-space baselines while using far fewer trainable parameters.

Prototype Steering achieved the strongest gains on the contiguous split, but its benefit depended on a coherent high-drift target group. Context Gate emerged as the most robust method overall: it remained stable across random seeds, transferred best beyond the high-drift subset, and was the fastest to train due to rapid convergence. Last Block FT could perform well when trained on representative data, but was more sensitive to data bias and optimization.

Overall, the results show that effective post-deployment adaptation does not require updating the full backbone. In this setting, small and targeted corrections to the learned sequence representation provide the best balance of accuracy, robustness, and efficiency.

6. Limitations

This study considers a controlled post-deployment adaptation setting, which has several limitations. First, evaluation is conducted in an overlap-item regime, so item cold-start effects are excluded. Second, the main comparison focuses on a filtered high-drift user subset, which may overstate gains relative to a mixed population with many stable users. Third, ranking is evaluated with 100 sampled negatives rather than full-catalogue ranking, which is standard but may not fully reflect deployment performance. Fourth, experiments are limited to a single backbone (SASRec), a single dataset (30Music), and a single-step adaptation setting.

References

Ahmadov, P. and Mansoury, M. From insight to intervention: Interpretable neuron steering for controlling popularity bias in recommender systems. *arXiv preprint arXiv:2601.15122*, 2026.

Chen, C., Geng, H., Yang, N., Yan, J., Xue, D., Yu, J., and Yang, X. Learning self-modulating attention in continuous time space with applications to sequential recommendation. In *International Conference on Machine Learning*, pp. 1606–1616. PMLR, 2021.

Chi, H., Xu, H., Fu, H., Liu, M., Zhang, M., Yang, Y., Hao, Q., and Wu, W. Long short-term preference modeling for continuous-time sequential recommendation. *arXiv preprint arXiv:2208.00593*, 2022.

Han, D., Huang, Y., Liu, J., Liao, K., and Lin, K. Lsab: User behavioral pattern modeling in sequential recommendation by learning self-attention bias. *ACM Transactions on Knowledge Discovery from Data*, 18(3):1–20, 2024.

Hao, W., Khan, G. A., Fan, G., and Zhang, C. Asrec: adaptive sequential recommendation with dynamic and periodic preferences capturing: W. hao et al. *Knowledge and Information Systems*, 68(1):6, 2026.

Kang, W., Kim, M., Lee, S., Kim, G.-W., and Choi, S.-M. Psr: A dual-level learning framework for patching sequential recommendation to capture local preference transitions. *IEEE Access*, 2025.

Kang, W.-C. and McAuley, J. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*, pp. 197–206. IEEE, 2018.

Klenitskiy, A., Volodkevich, A., Pembek, A., and Vasilev, A. Does it look sequential? an analysis of datasets for evaluation of sequential recommendations. In *Proceedings of the 18th ACM conference on recommender systems*, pp. 1067–1072, 2024.

Klenitskiy, A., Polev, K., Denisova, D., Vasilev, A., Simakov, D., and Gusev, G. Sparse autoencoders for sequential recommendation models: Interpretation and flexible control. *arXiv preprint arXiv:2507.12202*, 2025.

Lai, W., Fraser, A., and Titov, I. Joint localization and activation editing for low-resource fine-tuning. *arXiv preprint arXiv:2502.01179*, 2025.

Lee, G., Yoo, H., Hwang, J., Kang, S., and Yu, H. Leveraging historical and current interests for continual sequential recommendation. *arXiv e-prints*, pp. arXiv–2506, 2025.

Li, J., Wang, Y., and McAuley, J. Time interval aware self-attention for sequential recommendation. In *Proceedings of the 13th international conference on web search and data mining*, pp. 322–330, 2020.

Lin, X., Pan, W., and Ming, Z. Towards interest drift-driven user representation learning in sequential recommendation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 1541–1551, 2025.

Maneeroj, S. and Sritrakool, N. An end-to-end personalized preference drift aware sequential recommender system with optimal item utilization. *IEEE Access*, 10: 62932–62952, 2022.

Schirmer, M., Thielmann, A., Schwöbel, P., Martynec, T., Di Benedetto, G., London, B., and Stein, Y. Aligning recommendations with user popularity preferences. *arXiv preprint arXiv:2604.01036*, 2026.

Shao, W., Chen, X., Zhao, J., Xia, L., Zhang, J., and Yin, D. Sequential recommendation with user evolving preference decomposition. In *Proceedings of the Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*, pp. 253–263, 2023.

Song, X., Wang, X., Wang, H., Lu, H., Chen, Y., Ren, Z., and Chen, Z. Sa-caisr: Stage-adaptive and conflict-aware incremental sequential recommendation. *arXiv preprint arXiv:2602.08678*, 2026.

Sritrakool, N. and Maneeroj, S. Personalized preference drift aware sequential recommender system. *IEEE Access*, 9:155491–155506, 2021.

Turrin, R., Quadrana, M., Condorelli, A., Pagano, R., Cremonesi, P., et al. 30music listening and playlists dataset. *RecSys Posters*, 75, 2015.

Wu, Z., Arora, A., Wang, Z., Geiger, A., Jurafsky, D., Manning, C. D., and Potts, C. Reft: Representation finetuning for language models. *Advances in Neural Information Processing Systems*, 37:63908–63962, 2024.

Yin, F., Ye, X., and Durrett, G. Lofit: Localized fine-tuning on llm representations. *Advances in neural information processing systems*, 37:9474–9506, 2024.

A. SASRec backbone details

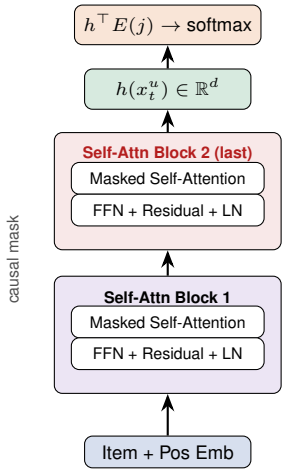


Figure 7. SASRec backbone (2 causal self-attention blocks): item and positional embeddings are encoded by masked self-attention and a point-wise FFN with residual connections and layer normalization; the final hidden state $h(x_t^u)$ scores items by dot product.

We use SASRec as the sequential recommendation backbone (Kang & McAuley, 2018) shown in Figure 7. SASRec models next-item prediction from an ordered sequence of user interactions using a Transformer-style self-attention encoder specialized for recommendation. Let the interaction history for user u be

$$S^u = (i_1^u, i_2^u, \ldots, i_{|S^u|}^u),$$

where each item belongs to a fixed catalog $\mathcal{I}$. The learning task is to predict the next item from a history prefix.

Sequence construction. Each history prefix is converted to a fixed-length input sequence of length n. If a prefix is longer than n, only the most recent n items are kept. If it is shorter than n, the sequence is left-padded with a special padding token. This matches the standard SASRec preprocessing, where the model focuses on the most recent segment of the interaction history (Kang & McAuley, 2018).

Input embeddings. Each item ID is mapped to a learned item embedding from

$$E_{\text{item}} \in \mathbb{R}^{(|\mathcal{I}|+1)\times d},$$

where index 0 corresponds to padding. A learned positional embedding

$$E_{\text{pos}} \in \mathbb{R}^{n\times d}$$

is added to encode order information. Thus, the input representation at position j is the sum of an item embedding and a positional embedding. Padding positions are masked so that they do not contribute to later computations.

Masked self-attention encoder. The embedded sequence is processed by a stack of self-attention blocks. In each block, the model first applies layer normalization, then multi-head self-attention with a causal mask, followed by a point-wise feed-forward network. Residual connections are used around both the attention and feed-forward sublayers, and dropout is applied for regularization. The causal mask ensures that when computing the representation at position t, the model can attend only to positions $1, \ldots, t$, preventing leakage from future items.

This design gives SASRec a key advantage over fixed-order Markov models and recurrent models. Instead of forcing the prediction to depend only on the last few items or compressing the whole history through a recurrent state, self-attention allows the model to adaptively weight whichever past items are most relevant for the current prediction. In sparse settings this often behaves like a short-range transition model, while in denser settings it can capture longer-range dependencies (Kang & McAuley, 2018).

Point-wise feed-forward layer. After attention, SASRec applies a point-wise feed-forward network independently at each sequence position. In the original SASRec formulation this is described as a two-layer feed-forward network. In our implementation, this module is realized using two 1×1 convolutions with a ReLU activation and dropout, which is functionally equivalent to applying the same small multilayer perceptron at every time step.

Sequence representation and prediction. After the final self-attention block, SASRec produces a contextualized hidden vector for each sequence position. In the original paper, the representation at each valid position can be used to predict the next item in a shifted-sequence training setup (Kang & McAuley, 2018). In our implementation, we extract only the final valid hidden state of the prefix sequence and use it as the sequence representation:

$$\mathbf{h}_{\text{last}} \in \mathbb{R}^d.$$

Candidate items are then scored by dot product with item embeddings. For item i, the score is

$$s(i \mid x) = \mathbf{h}_{\text{last}}^{\top} E_{\text{item}}(i).$$

Thus, the backbone produces a context-dependent representation of the recent user history, and the scoring layer converts this representation into next-item relevance scores.

Training objective in our implementation. The original SASRec paper trains with a binary cross-entropy objective using sampled negative items at each sequence position (Kang & McAuley, 2018). Our implementation differs slightly. Rather than applying token-level supervision at all positions with sampled negatives, we create one training example for each prefix-target pair in a user sequence. For a sequence

$$(i_1^u, i_2^u, \ldots, i_T^u),$$

we form examples

$$(i_1^u \to i_2^u),\ (i_1^u, i_2^u \to i_3^u),\ \ldots,\ (i_1^u, \ldots, i_{T-1}^u \to i_T^u).$$

For each example, the model extracts the final valid hidden state of the prefix, scores the full item vocabulary, and applies multiclass cross-entropy on the observed next item. If y denotes the ground-truth next item, the loss for one example is

$$\mathcal{L} = -\log \frac{\exp(s(y \mid x))}{\sum_{j \in \mathcal{I}} \exp(s(j \mid x))}.$$

This full-softmax next-item classification objective is the loss actually used to train our backbone. It differs from the original SASRec objective, but the underlying encoder architecture remains SASRec.

Implementation settings. Our backbone uses hidden size $d = 64$, two self-attention blocks, two attention heads, maximum sequence length 128, dropout, and Adam optimization, following the lightweight configuration used in *Does It Look Sequential?* (Klenitskiy et al., 2024). We disable the token-level output head during backbone training and instead score items from the final sequence representation, as described above.

B. Dataset construction and filtering pipeline

B.1. Temporal split construction

Let the full interaction log be sorted chronologically, and let the selected interaction budget be N. We define

$$|t_1| = \lfloor 0.5N \rfloor, \qquad |t_2| = N - |t_1|.$$

The historical block t_1 is always the first half of the selected interactions. We construct two future-period regimes:

Contiguous split. The future block t_2 is the chronological block immediately following t_1.

Tail split. The future block t_2 is the final chronological block of the dataset.

Thus, both regimes share the same t_1 but differ in the temporal gap between t_1 and t_2. In each case, t_2 is divided chronologically into

$$t_2 = t_2^{\text{adapt}} \cup t_2^{\text{test}}, \qquad |t_2^{\text{adapt}}| : |t_2^{\text{test}}| = 70 : 30.$$

B.2. Overlap-item regime

A large fraction of items appearing in t_2 may be absent from t_1. If such out-of-vocabulary items are left in place, the observed performance drop is dominated by catalogue expansion rather than behavioural shift. To isolate temporal drift over previously seen items, we evaluate in an overlap-item regime.

We first apply iterative 5-core filtering to t_1. That is, users with fewer than 5 interactions and items with fewer than 5 occurrences are removed repeatedly until convergence. Let $\mathcal{I}_{\text{hist}}$ denote the resulting historical item set. We then restrict both t_2^{adapt} and t_2^{test} to interactions whose items lie in $\mathcal{I}_{\text{hist}}$.

B.3. High-drift user selection

Even within the overlap-item regime, many users remain relatively stable across t_1 and t_2. To focus evaluation on users whose behaviour changes materially over time, we compute a drift score and retain a high-drift subset.

Let $\mathcal{U}_{\text{ov}}$ be the users present in both overlap-filtered t_1 and t_2^{adapt}, with sufficient sequence length in both periods. For each $u \in \mathcal{U}_{\text{ov}}$, let

$$\mathbf{s}_u^{(1)} = (i_{u,1}^{(1)}, \ldots, i_{u,T_u^{(1)}}^{(1)}), \qquad \mathbf{s}_u^{(2)} = (i_{u,1}^{(2)}, \ldots, i_{u,T_u^{(2)}}^{(2)})$$

denote the chronological item sequences in t_1 and t_2, respectively.

We define three user-level drift signals.

Item-profile drift. Let $\mathbf{c}_u^{(1)}$ and $\mathbf{c}_u^{(2)}$ be item-count vectors over the union of items observed for user u across both periods. We define

$$d_u^{\text{item}} = 1 - \frac{\langle \mathbf{c}_u^{(1)}, \mathbf{c}_u^{(2)} \rangle}{\|\mathbf{c}_u^{(1)}\|_2 \, \|\mathbf{c}_u^{(2)}\|_2}.$$

Hidden-state drift. Let f_θ denote the frozen SASRec backbone trained on overlap-filtered t_1. For each period, we extract the final sequence representation

$$\mathbf{h}_u^{(1)} = f_\theta(\mathbf{s}_u^{(1)}), \qquad \mathbf{h}_u^{(2)} = f_\theta(\mathbf{s}_u^{(2)}),$$

using truncation to the model maximum length and left-padding when required. We then define

$$d_u^{\text{hid}} = 1 - \frac{\langle \mathbf{h}_u^{(1)}, \mathbf{h}_u^{(2)} \rangle}{\|\mathbf{h}_u^{(1)}\|_2 \, \|\mathbf{h}_u^{(2)}\|_2}.$$

Top-k item drift. Let $A_u^{(1)}$ and $A_u^{(2)}$ be the sets of the top-k most frequent items for user u in t_1 and t_2, respectively. We define

$$d_u^{\text{top}k} = 1 - \frac{|A_u^{(1)} \cap A_u^{(2)}|}{|A_u^{(1)} \cup A_u^{(2)}|}.$$

B.4. Combined drift score

Because the three drift signals lie on different scales, we standardize each across users. For

$$j \in \{\text{item}, \text{hid}, \text{top}k\},$$

let

$$z_{u,j} = \frac{d_{u,j} - \mu_j}{\sigma_j},$$

where μ_j and σ_j are the mean and standard deviation of metric j over $\mathcal{U}_{\text{ov}}$. The combined drift score is then

$$d_u^{\text{comb}} = \frac{1}{3}\left(z_u^{\text{item}} + z_u^{\text{hid}} + z_u^{\text{top}k}\right).$$

Users are ranked by d_u^{comb}, and the top fraction is retained as the high-drift subset.

B.5. Final filtered subset

After selecting the high-drift users, we restrict the overlap-filtered historical and future splits to this user subset. We then re-apply iterative 5-core filtering to the historical split and again restrict the future splits to the resulting historical item set. This produces the final high-drift overlap-item evaluation subset used in the main experiments.

B.6. User clustering for prototype steering

Prototype steering assigns each user to one of K prototype groups before training the cluster-specific adapters. The clustering is performed in a low-dimensional drift-feature space.

For each user u, we define the feature vector

$$\mathbf{x}_u = \begin{bmatrix} d_u^{\text{item}} \\ d_u^{\text{hid}} \\ d_u^{\text{top}k} \\ \ell_u^{\text{hist}} \\ \ell_u^{\text{future}} \end{bmatrix},$$

where ℓ_u^{hist} and ℓ_u^{future} denote historical and future sequence lengths when included.

Each feature is standardized across users:

$$x'_{u,j} = \frac{x_{u,j} - \mu_j}{\sigma_j}.$$

The standardized vectors $\mathbf{x}'_u$ are then clustered with KMeans into K groups. The resulting assignment

$$z_u \in \{0, \ldots, K-1\}$$

determines which prototype-specific residual adapter is applied to user u in prototype steering.

B.7. Cluster-size distributions for Prototype Steering

Table 5. Cluster-size distributions (K) for high-drift users.

Split	K	Users	%
Contiguous (794 users)	$K = 3$	582	73.3%
		208	26.2%
		4	0.5%
	$K = 5$	557	70.2%
		149	18.8%
		77	9.7%
		7	0.9%
		4	0.5%
	$K = 10$	305	38.4%
		170	21.4%
		97	12.2%
		76	9.6%
		49	6.2%
		41	5.2%
		32	4.0%
		14	1.8%
		6	0.8%
		4	0.5%
Tail (953 users)	$K = 3$	621	65.2%
		229	24.0%
		103	10.8%
	$K = 5$	598	62.7%
		199	20.9%
		86	9.0%
		62	6.5%
		8	0.8%
	$K = 10$	345	36.2%
		247	25.9%
		94	9.9%
		82	8.6%
		82	8.6%
		69	7.2%
		20	2.1%
		9	0.9%
		4	0.4%
		1	0.1%

Table 5 reports the cluster-size distributions.

C. Sequential-structure sanity check

Motivation. Overlap-item filtering and high-drift user selection may reduce the sequential signal in the data. Since our study evaluates sequential recommenders, it is important to verify that the filtered subsets still exhibit meaningful order-dependent structure.

Method. We follow the sequential-rules approach of Klenitskiy et al. (2024). Let user sequences be of the form

$$(i_1, i_2, \ldots, i_t).$$

We count sequential rules based on short contiguous patterns:

$$(i_{t-1}) \to i_t \quad \text{(2-grams)},$$

$$(i_{t-2}, i_{t-1}) \to i_t \quad \text{(3-grams)}.$$

To assess whether these patterns reflect genuine sequential structure, we construct a shuffled version of the data by randomly permuting each user's prefix while keeping the target item fixed. This destroys order information but preserves item frequency and marginal distributions.

Let $C^{(L)}_{\text{orig}}$ and $C^{(L)}_{\text{shuf}}$ denote the counts of valid L-gram rules before and after shuffling. We measure the relative change

$$\Delta^{(L)} = \frac{C^{(L)}_{\text{shuf}} - C^{(L)}_{\text{orig}}}{C^{(L)}_{\text{orig}}}.$$

A large negative value of $\Delta^{(L)}$ indicates that most sequential rules disappear after shuffling, implying strong order-dependent structure.

Results. We apply this procedure to the raw, overlap-filtered, and high-drift subsets for both temporal regimes. Table 6 summarizes the results.

Table 6. Sequential-structure sanity check results.

Split	Subset	$\Delta^{(2)}$	$\Delta^{(3)}$	Verdict
Contiguous	Raw future test	-92.9%	-95.9%	Strong
Contiguous	Overlap + 5-core	-92.5%	-95.1%	Strong
Contiguous	High-drift 5-core	-76.8%	-92.3%	Moderate
Tail	Raw future test	-93.9%	-95.9%	Strong
Tail	Overlap + 5-core	-94.0%	-96.3%	Strong
Tail	High-drift 5-core	-62.3%	-66.9%	Weak

Discussion. Overlap-item filtering preserves strong sequential structure, with rule collapse comparable to the raw dataset. In contrast, restricting to high-drift users weakens sequential patterns, especially in the tail regime. This reflects the fact that large behavioural shifts often disrupt stable local transition structure.

Overall, the filtered subsets retain sufficient sequential signal to justify the use of SASRec, while also introducing controlled distribution shift for evaluating adaptation methods.

D. Hyperparameter Search

D.1. Search space

Table 7 lists the hyperparameter grids used for each adaptation method. The best configuration per method and split was selected by NDCG@10 on the test set.

Table 7. Hyperparameter search spaces. Total configs is the Cartesian product of all values for each method.

Method	Hyperparam	Values	Total
Last Block FT	lr	1e-3, 5e-4, 1e-4, 5e-5	32
	epochs	20, 50	
	include_last_layernorm	False, True	
	weight_decay	0.0, 1e-4	
Context Gate	lr	1e-3, 5e-4, 1e-4	24
	epochs	10, 20	
	bottleneck_dim	4, 8, 16, 32	
Prototype Steering	lr	1e-3, 5e-4	60
	epochs	20, 50	
	num_clusters	1, 3, 5, 10, 20	
	bottleneck_dim	16, 32, 64	

D.2. Sweep-level analysis

Best configurations and the cluster-count sensitivity analysis are reported in Table 8. Figures 8–11 show the full sweep results.

Table 8. Best hyperparameter configuration per method and split, selected by NDCG@10. bd = bottleneck dimension, ln = include last layer norm, wd = weight decay.

Method	Split	Best Configuration	NDCG@10 %Δ
Last Block FT	Contiguous	lr=5e-4, ep=50, ln=True, wd=0.0	+4.48%
	Tail	lr=1e-3, ep=50, ln=True, wd=1e-4	+8.55%
Context Gate	Contiguous	bd=8, lr=5e-4, ep=10	+7.10%
	Tail	bd=32, lr=1e-3, ep=20	+10.39%
Prototype Steering	Contiguous	K=1, bd=16, lr=5e-4, ep=20	+4.58%
	Tail	K=5, bd=64, lr=1e-3, ep=20	+12.56%

Figure 8 shows the spread of NDCG@10 across every run in each sweep. Context Gate (Tail) and Prototype Steering (Tail) have the highest medians and widest upside range. Prototype Steering (Contiguous) is the only case where a substantial fraction of runs fall below zero, driven entirely by the $K \geq 5$ configurations (Figure 11).

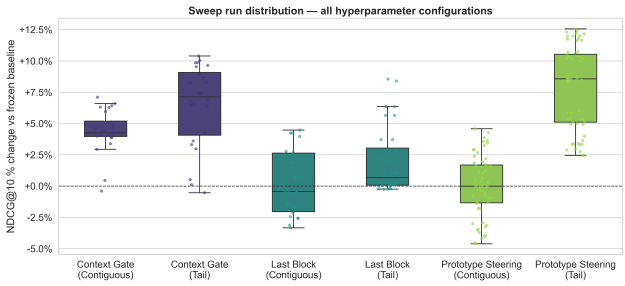
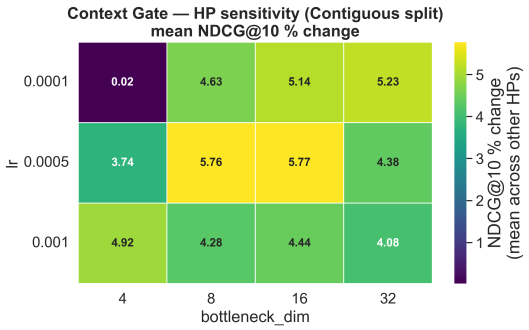
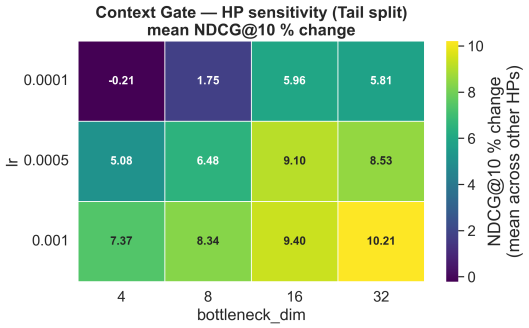


Figure 8. Distribution of NDCG@10 improvement (%) across *all* hyperparameter configurations in the sweep (not just the best run). Context Gate (Tail) and Prototype Steering (Tail) show the highest median and widest upside range. Prototype Steering (Contiguous) has a wide distribution crossing zero, confirming that poor cluster counts ($K \geq 5$) actively hurt performance on that split.

Context Gate. The dominant axis is learning rate: lr=1e-4 nearly eliminates all gain on the contiguous split (best cell +0.02%). On the tail split the optimal shifts to lr=1e-3 with larger bottleneck (bd=32, +10.21%), confirming that stronger drift benefits from higher capacity and more aggressive updates.



(a) Contiguous split

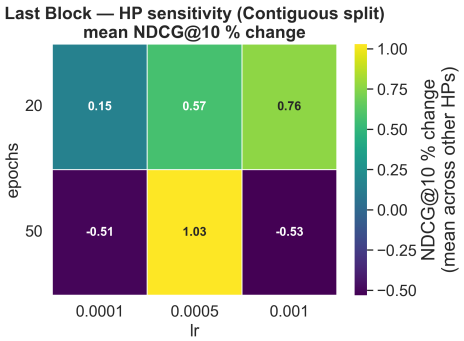


(b) Tail split

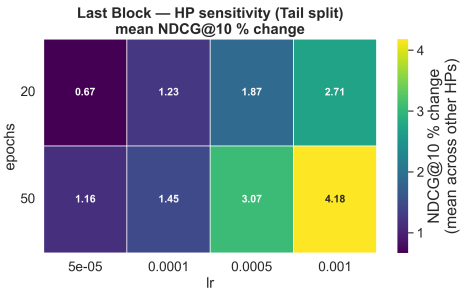
Figure 9. Context Gate HP sensitivity. On the contiguous split, lr=5e-4 with bd $\in \{8, 16\}$ is optimal. On the tail split, the sweet spot shifts toward lr=1e-3 and larger bottleneck dimensions (bd=32), reflecting the benefits of higher-capacity adapters under stronger drift.

Last Block FT. Epoch count interacts strongly with learning rate. On the contiguous split, ep=50 with lr=5e-4 is the only combination that reliably improves (+1.03%); lr=1e-3 at ep=50 is harmful (-0.53%), suggesting the backbone overfits quickly at high learning rates. On the tail split the interaction softens and higher learning rates are tolerated at 50 epochs (+4.18% at lr=1e-3).

Prototype Steering. The cluster count K dominates and learning rate has comparatively little effect once K is fixed. On the contiguous split, $K = 1$ with either learning rate yields positive gains (+2.6–3.6%); all $K \geq 5$ configurations are harmful. On the tail split, $K = 5$ is the peak regardless of learning rate (+12.0–12.1%), with performance degrading symmetrically on either side.



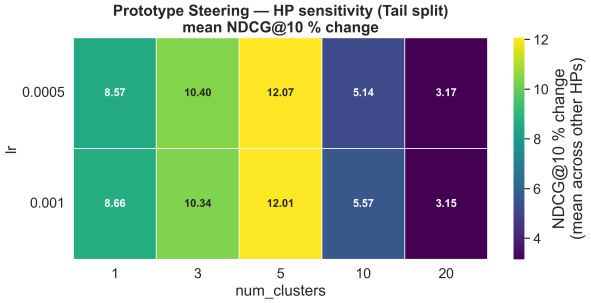
(a) Contiguous split



(b) Tail split

Figure 10. Last Block FT HP sensitivity: mean NDCG@10 improvement (%) as a function of epochs and learning rate. On the contiguous split, ep=50 and lr=5e-4 is optimal (+1.03%); higher learning rates or lower epoch counts often prove harmful. In contrast, the tail split shows consistent gains at ep=50 across all learning rates.

(a) Contiguous split



(b) Tail split

Figure 11. Prototype Steering HP sensitivity: mean NDCG@10 improvement (%) as a function of learning rate and number of clusters K. The cluster-count effect dominates: performance degrades monotonically on the contiguous split from $K = 1$ to $K = 20$, while it follows an inverted-U trend on the tail split peaking at $K = 5$. Learning rate has a secondary effect compared to K.